



MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

CINEVAULT RENTALS



MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

S.No.	Section / Topic	Questions	Page No.
1	Section A – Schema Design & DDL	Q1-Q4	2 -7
2	Section B – Data Manipulation	Q5-Q7	7- 10
3	Section C – Business Filtering & Operators	Q8-Q10	10- 15
4	Section D – Relationship Analysis: All Joins	Q11-Q15	15-21
5	Section E – Aggregation, Grouping & Output-Based Questions	Q16-Q18	21-25
6	Section F – Advanced Analytics: Subqueries & Window Functions	Q19-Q20	26-31
7	Bonus – CTE, JOIN & Window Function	-----	31-33

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

SECTION A: Schema Design & DDL

Q1. New Franchise Rollout

a) Create cv_employee_perf

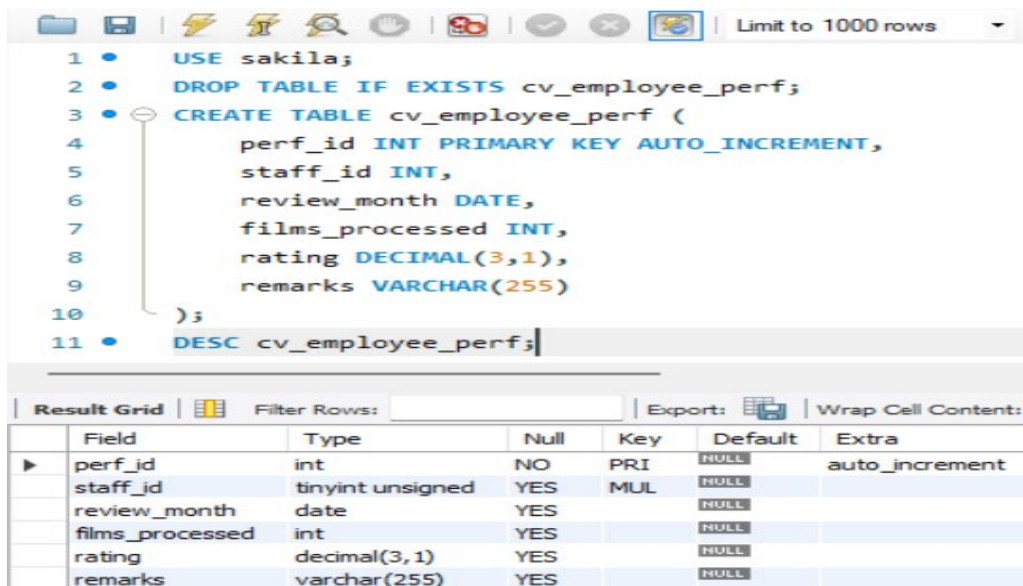
Query : USE sakila;

DROP TABLE IF EXISTS cv_employee_perf;

```
CREATE TABLE cv_employee_perf (  
    perf_id INT PRIMARY KEY AUTO_INCREMENT,  
    staff_id INT,  
    review_month DATE,  
    films_processed INT,  
    rating DECIMAL(3,1),  
    remarks VARCHAR(255)  
);
```

Verify the table structure

Query : DESC cv_employee_perf;



The screenshot shows a MySQL IDE window with a SQL query editor and a result grid. The query editor contains the following SQL code:

```
1 • USE sakila;  
2 • DROP TABLE IF EXISTS cv_employee_perf;  
3 • CREATE TABLE cv_employee_perf (  
4     perf_id INT PRIMARY KEY AUTO_INCREMENT,  
5     staff_id INT,  
6     review_month DATE,  
7     films_processed INT,  
8     rating DECIMAL(3,1),  
9     remarks VARCHAR(255)  
10 );  
11 • DESC cv_employee_perf;
```

The result grid displays the table structure for cv_employee_perf. The columns are Field, Type, Null, Key, Default, and Extra. The rows are:

Field	Type	Null	Key	Default	Extra
perf_id	int	NO	PRI	NULL	auto_increment
staff_id	tinyint unsigned	YES	MUL	NULL	
review_month	date	YES		NULL	
films_processed	int	YES		NULL	
rating	decimal(3,1)	YES		NULL	
remarks	varchar(255)	YES		NULL	

b) Add a FOREIGN KEY on staff_id referencing staff(staff_id).

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

the performance table is created, we need to connect it with the existing staff table. We use staff_id for this connection. By making it a foreign key, SQL ensures that a performance record can only refer to an employee who actually exists in the staff table.

Query : ALTER TABLE cv_employee_perf

ADD CONSTRAINT fk_cv_employee_perf_staff

FOREIGN KEY (staff_id)

REFERENCES staff(staff_id);

c) Management realizes rating should never be NULL and should default to 3.0. Alter the table accordingly

Management has now decided that every employee performance record must have a rating. we modify the existing rating column and add NOT NULL, so it cannot be left as NULL. We also add DEFAULT 3.0, so if someone inserts a performance record without specifying a rating, MySQL automatically stores 3.0.

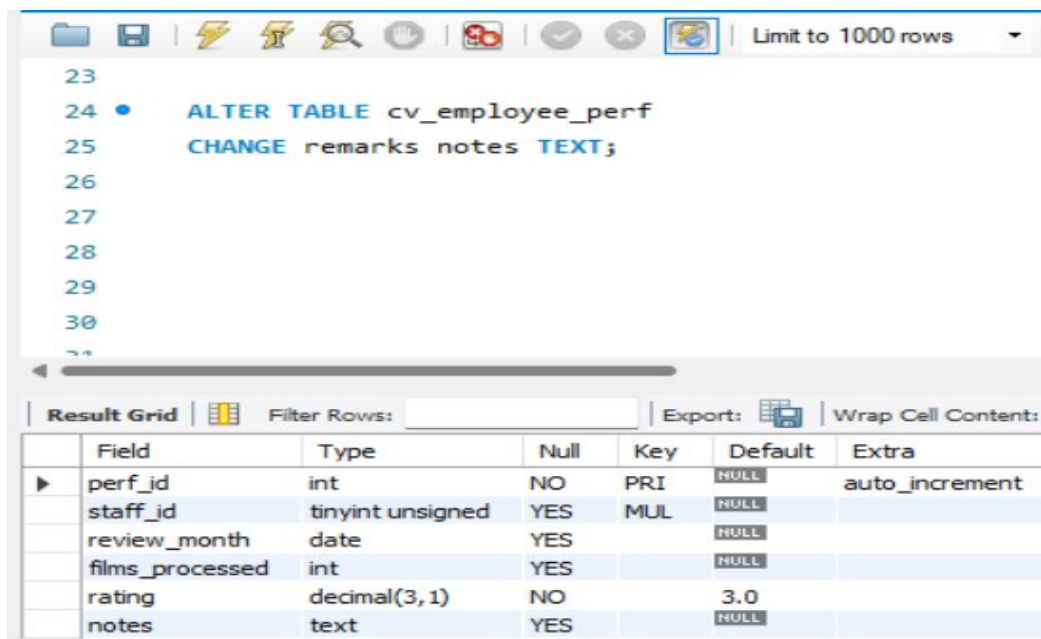
Query : ALTER TABLE cv_employee_perf

MODIFY rating DECIMAL(3,1) NOT NULL DEFAULT 3.0;

d) Rename remarks to notes and change VARCHAR(255) into TEXT

Query : ALTER TABLE cv_employee_perf

CHANGE remarks notes TEXT;



The screenshot shows a MySQL database management tool interface. At the top, there's a toolbar with various icons. Below it, a query editor displays the following SQL commands:

```
23
24 • ALTER TABLE cv_employee_perf
25   CHANGE remarks notes TEXT;
26
27
28
29
30
31
```

Below the query editor, there's a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid itself is a table with the following columns: Field, Type, Null, Key, Default, and Extra.

Field	Type	Null	Key	Default	Extra
perf_id	int	NO	PRI	NULL	auto_increment
staff_id	tinyint unsigned	YES	MUL	NULL	
review_month	date	YES		NULL	
films_processed	int	YES		NULL	
rating	decimal(3,1)	NO		3.0	
notes	text	YES		NULL	

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

Q2. Data Type Conversion Issue

a) ALTER TABLE statement to widen rental_rate to DECIMAL(6,2) without losing existing data

The existing rental_rate column is DECIMAL(4,2), which is not sufficient for the new premium pricing of up to 999.99. So, we use ALTER TABLE with MODIFY to change the column to DECIMAL(6,2). This increases the allowed size of the column while keeping the existing decimal format and data.

Query : ALTER TABLE film

MODIFY rental_rate DECIMAL(6,2);

Verify The New Data

DESC film;

```
27 • ALTER TABLE film
28   MODIFY rental_rate DECIMAL(6,2);
29
30 • DESC film;
31
```

Field	Type	Null	Key	Default	Extra
film_id	smallint unsigned	NO	PRI	NULL	auto_increment
title	varchar(128)	NO	MUL	NULL	
description	text	YES		NULL	
release_year	year	YES		NULL	
language_id	tinyint unsigned	NO	MUL	NULL	
original_language_id	tinyint unsigned	YES	MUL	NULL	
rental_duration	tinyint unsigned	NO		3	
rental_rate	decimal(6,2)	YES		NULL	
length	smallint unsigned	YES		NULL	
replacement_cost	decimal(5,2)	NO		19.99	
rating	enum('G','PG','P...)	YES		G	
special_features	set('Trailers','Co...)	YES		NULL	
last_update	timestamp	NO		CURR...	DEFAULT_GEN...

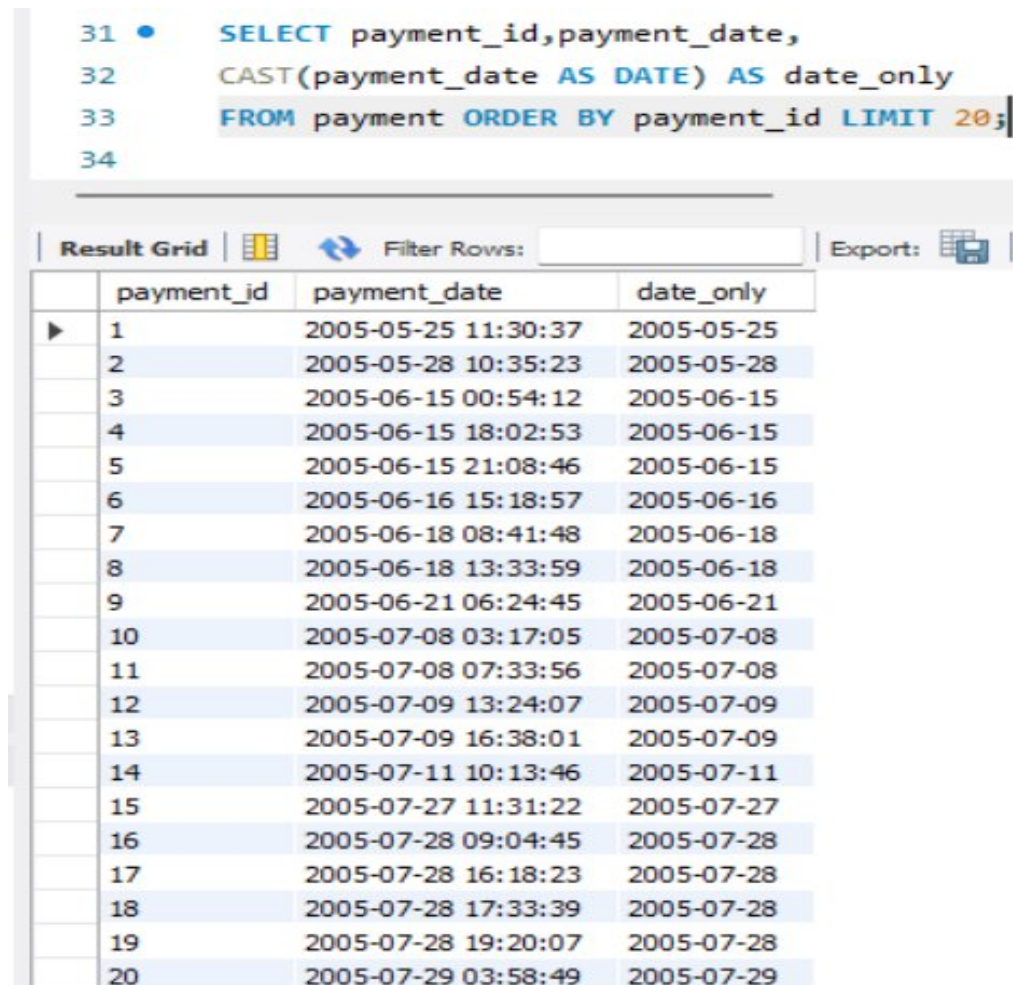
DESC film is used to verify the structure of the film table after the alteration. It allows us to confirm that the rental_rate column has been successfully changed from DECIMAL(4,2) to DECIMAL(6,2) without affecting the existing data.

b) Write a query to convert and display the payment_date (DATETIME) from the payment table as just a DATE value using a conversion function.

The query uses CAST(payment_date AS DATE) to convert the DATETIME value into a DATE value for display, removing the time component without changing the actual column in the table. ORDER BY sorts the results by payment ID, and LIMIT 20 displays the first 20 records.

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

Query : SELECT payment_id,payment_date,
CAST(payment_date AS DATE) AS date_only
FROM payment ORDER BY payment_id LIMIT 20;



```
31 • SELECT payment_id,payment_date,  
32     CAST(payment_date AS DATE) AS date_only  
33     FROM payment ORDER BY payment_id LIMIT 20;  
34
```

	payment_id	payment_date	date_only
▶	1	2005-05-25 11:30:37	2005-05-25
	2	2005-05-28 10:35:23	2005-05-28
	3	2005-06-15 00:54:12	2005-06-15
	4	2005-06-15 18:02:53	2005-06-15
	5	2005-06-15 21:08:46	2005-06-15
	6	2005-06-16 15:18:57	2005-06-16
	7	2005-06-18 08:41:48	2005-06-18
	8	2005-06-18 13:33:59	2005-06-18
	9	2005-06-21 06:24:45	2005-06-21
	10	2005-07-08 03:17:05	2005-07-08
	11	2005-07-08 07:33:56	2005-07-08
	12	2005-07-09 13:24:07	2005-07-09
	13	2005-07-09 16:38:01	2005-07-09
	14	2005-07-11 10:13:46	2005-07-11
	15	2005-07-27 11:31:22	2005-07-27
	16	2005-07-28 09:04:45	2005-07-28
	17	2005-07-28 16:18:23	2005-07-28
	18	2005-07-28 17:33:39	2005-07-28
	19	2005-07-28 19:20:07	2005-07-28
	20	2005-07-29 03:58:49	2005-07-29

Q3. Decommissioning & Cleanup

- a) Write a query to remove all rows from cv_employee_perf but keep the table structure intact for future use.

The requirement is to **remove all the existing performance records** from cv_employee_perf, but the company may want to use the table again in the future, So we use TRUNCATE TABLE. It removes **all rows/data** from the table but **does not remove the table structure**.

Query : TRUNCATE TABLE cv_employee_perf;

Verification

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

```
329 • TRUNCATE TABLE cv_employee_perf;  
330 • SELECT *  
331 FROM cv_employee_perf;  
332 • DESC cv_employee_perf;  
333  
334
```

Field	Type	Null	Key	Default	Extra
perf_id	int	NO	PRI	NULL	auto_increment
staff_id	tinyint unsigned	YES	MUL	NULL	
review_month	date	YES		NULL	
films_processed	int	YES		NULL	
rating	decimal(3,1)	NO		3.0	
notes	text	YES		NULL	

- b) Six months later, management confirms the module is cancelled permanently.
Write the query to remove the table entirely

management has confirmed that the Employee Performance module is permanently cancelled, so the table is no longer needed. We use DROP TABLE because it removes the entire cv_employee_perf table, including both its data and table structure.

Query : DROP TABLE cv_employee_perf;

Verification:

```
334 • DROP TABLE cv_employee_perf;  
335 • SHOW TABLES LIKE 'cv_employee_perf';  
336
```

Result Grid	Filter Rows:	Export:
Tables_in_sakila (cv_employee_perf)		

- c) explain why you'd choose TRUNCATE for (a) and DROP for (b) instead of using DELETE for both.

Explanation: We choose **TRUNCATE** in (a) because we want to remove all rows but **keep the table structure** for future use. We choose **DROP** in (b) because the module is permanently cancelled, so we want to remove the **entire table and its structure**. DELETE is mainly used when we want to remove specific rows using a WHERE condition.

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

Temporary cleanup → TRUNCATE

Permanent removal → DROP

Q4. Normalization Check

- a) Identify which Normal Form this design violates and explain briefly why storing repeating customer/film details in every rental row is a problem.**

Explanation: The flat table repeats the same customer and film information for every rental, creating data redundancy. This can cause update, insertion, and deletion anomalies; for example, if a customer's email changes, it may need to be updated in many rows.

- b) Using the existing Sakila schema as your reference, briefly explain how Sakila already avoids this problem — name the tables/keys involved.**

Explanation : Sakila separates the information into different related tables. The customer table stores customer details using customer_id, while film stores film details using film_id; inventory connects a film using film_id, and rental connects the customer using customer_id and the rented inventory using inventory_id.

Normalization --> Store each piece of information in the right place and connect tables using keys.

SECTION – B: Data Manipulation

Q5. Onboarding New Staff

first_name, last_name, address_id, store_id, active, username, picture and password may be NULL; email may be NULL. Address_id = 3 and store_id = 1 exist in the supplied Sakila data. The column list is explicit as required.

Operations has hired a new employee, so we need to add a new record to the existing staff table. We use INSERT INTO because the requirement is to add a new row.

Query : INSERT INTO staff

(first_name, last_name, address_id, email, store_id, active, username, password) VALUES

('ANANYA', 'RAO', 1, 'ananya@cinevault.com', 1, 1, 'ananya', 'ananya123');

--- Verify the new data is inserted or not

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

Query : `SELECT * FROM staff ORDER BY staff_id DESC LIMIT 1;`

```
41
42 • SELECT * FROM staff ORDER BY staff_id DESC LIMIT 1;
43
44
```

staff_id	first_name	last_name	address_id	picture	email	store_id	active	username	password	last_update
3	ANANYA	RAO	1		ananya@cinevault.com	1	1	ananya	ananya123	2026-09-10 14:34:04

Q6. Price Correction — Bulk Update

a) Write an **UPDATE** query to increase the **rental_rate** of all 'PG' rated films by 15%, rounded to 2 decimal places.

Increase the rental rate of all PG-rated films by 15%. ROUND is used to keep the new rental rate to 2 decimal places. The WHERE condition ensures that only films with a PG rating are updated

Query : `UPDATE film SET rental_rate = ROUND(rental_rate * 1.15, 2)`

`WHERE rating = 'PG';`

```
48 • SET SQL_SAFE_UPDATES = 1;
49 -- verify the rows
50 • SELECT film_id, title, rating, rental_rate
51 FROM film
52 WHERE rating = 'PG';
```

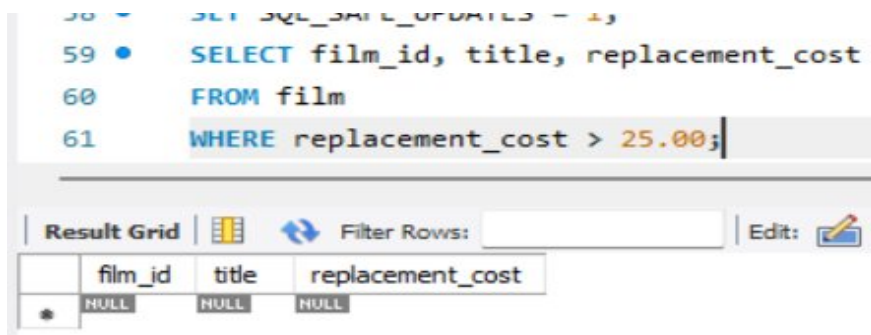
film_id	title	rating	rental_rate
1	ACADEMY DINOSAUR	PG	1.14
6	AGENT TRUMAN	PG	3.44
12	ALASKA PHANTOM	PG	1.14
13	ALI FOREVER	PG	5.74
19	AMADEUS HOLY	PG	1.14
37	ARIZONA BANG	PG	3.44
41	ARSENIC INDEPENDENCE	PG	1.14
63	BEDAZZLED MARRIED	PG	1.14
65	BEHAVIOR RUNAWAY	PG	5.74
72	BILL OTHERS	PG	3.44
74	BIRCH ANTITRUST	PG	5.74
78	BLACKOUT PRIVATE	PG	3.44
84	BOILED DARES	PG	5.74
88	BORN SPINAL	PG	5.74
91	BOUND CHEAPER	PG	1.14
93	BRANNIGAN SUNRISE	PG	5.74
95	BRINGING OUT THE DEAD	PG	2.44

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

- b) Finance now wants to cap the replacement_cost at 25.00 for any film currently priced above that — write the query.

First, `SQL_SAFE_UPDATES = 0` temporarily disables MySQL Safe Update Mode so that the required `UPDATE` operation can be performed without being blocked. The `UPDATE` statement then changes the replacement_cost to 25.00 only for films whose current replacement cost is greater than 25.00, as specified by the `WHERE` condition. After completing the update, `SQL_SAFE_UPDATES = 1` is used to turn Safe Update Mode back on and restore the protection against accidental updates.

```
Query : SET SQL_SAFE_UPDATES=0;
UPDATE film
SET replacement_cost = 25.00
WHERE replacement_cost > 25.00;
SET SQL_SAFE_UPDATES = 1;
```



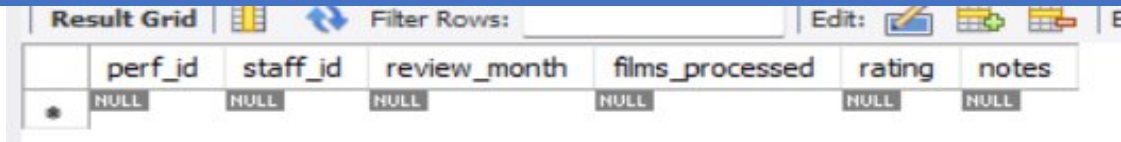
Q7. Data Cleanup — Conditional Delete

- a) Delete the test/junk rows

First, `SQL_SAFE_UPDATES = 0` temporarily disables Safe Update Mode so the `DELETE` operation can be executed. The `DELETE` statement removes only those rows where `films_processed` is 0 and the `rating` is less than 2.0; the `AND` means both conditions must be true. After deletion, Safe Update Mode is turned back on using `SQL_SAFE_UPDATES = 1`. Finally, the `SELECT` query verifies whether any matching rows are still present; ideally, it should return no rows.

```
Query : SET SQL_SAFE_UPDATES=0;
DELETE FROM cv_employee_perf
WHERE films_processed = 0
AND rating < 2.0;
SET SQL_SAFE_UPDATES=1;
SELECT * FROM cv_employee_perf WHERE films_processed = 0 AND
rating < 2.0;
```

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS



	perf_id	staff_id	review_month	films_processed	rating	notes
*	NULL	NULL	NULL	NULL	NULL	NULL

- b) Explain what safeguard you should apply before running a DELETE without a WHERE clause in production.

Explanation: Before running a DELETE without a WHERE clause in production, **take a backup and first verify the affected rows using a SELECT query**. Ideally, test the operation in a transaction/staging environment before permanently deleting the data.

Section C — Business Filtering & Operators

Q8. Marketing Segment 1 — Multi-condition Filter

Get active customers living in California or Texas, with customer IDs from 1 to 200. The customer and address tables are joined using address_id.

```
Query : SELECT c.customer_id,  
        c.first_name,  
        c.last_name,  
        c.email,  
        a.district  
FROM customer c  
JOIN address a ON c.address_id = a.address_id  
WHERE c.active = 1  
      AND a.district IN ('California', 'Texas')  
      AND c.customer_id BETWEEN 1 AND 200;
```

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

```
71     AND a.district IN ('California', 'Texas')
72     AND c.customer_id BETWEEN 1 AND 200;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	customer_id	first_name	last_name	email	district
▶	2	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org	California
	6	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org	Texas
	14	BETTY	WHITE	BETTY.WHITE@sakilacustomer.org	California
	51	ALICE	STEWART	ALICE.STEWART@sakilacustomer.org	California
	112	ROSA	REYNOLDS	ROSA.REYNOLDS@sakilacustomer.org	California
	118	KIM	CRUZ	KIM.CRUZ@sakilacustomer.org	Texas
	182	RENEE	LANE	RENEE.LANE@sakilacustomer.org	California

Explanation: We join the customer and address tables using address_id because the customer's district is stored in the address table. The WHERE condition uses AND to ensure all requirements are satisfied: the customer must be active, the district must be either California or Texas using IN, and the customer_id must be between 1 and 200 using BETWEEN.

Q9. Marketing Segment 2 — Pattern & NULL Handling

- a) Write a query to find all customers whose email contains 'sakilacustomer.org'

Query : **SELECT customer_id, first_name, last_name, email**
FROM customer
WHERE email LIKE '%sakilacustomer.org%';

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

Result Grid   Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 

	customer_id	first_name	last_name	email
▶ 1	MARY	SMITH	MARY.SMITH@sakilacustomer.org	
2	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org	
3	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	
4	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	
5	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org	
6	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org	
7	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	
8	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org	
9	MARGARET	MOORE	MARGARET.MOORE@sakilacustomer.org	
10	DOROTHY	TAYLOR	DOROTHY.TAYLOR@sakilacustomer.org	
11	LISA	ANDERSON	LISA.ANDERSON@sakilacustomer.org	
12	NANCY	THOMAS	NANCY.THOMAS@sakilacustomer.org	
13	KAREN	JACKSON	KAREN.JACKSON@sakilacustomer.org	
14	BETTY	WHITE	BETTY.WHITE@sakilacustomer.org	
15	HELEN	HARRIS	HELEN.HARRIS@sakilacustomer.org	

customer 14 ×

Output

 Action Output

#	Time	Action	Message
✓ 137	14:43:36	UPDATE film SET replacement_cost = 25.00 WHERE replacement_cost ...	236 row(s) affected Rows matched: 236
✓ 138	14:43:36	SET SQL_SAFE_UPDATES = 1	0 row(s) affected
✓ 139	14:44:18	SELECT film_id, title, replacement_cost FROM film WHERE replacement_...	0 row(s) returned
✓ 140	14:49:25	SELECT c.customer_id, c.first_name, c.last_name, c.email, ...	7 row(s) returned
✓ 141	14:53:17	SELECT customer_id, first_name, last_name, email FROM c...	599 row(s) returned

Explanation : We use LIKE to search for a particular pattern inside the email column. The % symbols mean that any characters can appear before or after sakilacustomer.org, so the query returns customers whose email contains that text anywhere in the email address.

b) Write a query to find all films where the description does not contain the word 'Amazing'.

NOT LIKE excludes descriptions containing 'Amazing'. '%' allows any characters before or after the word. ORDER BY film_id displays the results in film ID order.

Query : SELECT film_id, title, description

FROM film

WHERE description NOT LIKE '%Amazing%' ORDER BY film_id;

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	film_id	title	description
▶	1	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad Scientist ...
	2	ACE GOLDFINGER	A Astounding Epistle of a Database Administrat...
	3	ADAPTATION HOLES	A Astounding Reflection of a Lumberjack And a ...
	4	AFFAIR PREJUDICE	A Fanciful Documentary of a Frisbee And a Lum...
	5	AFRICAN EGG	A Fast-Paced Documentary of a Pastry Chef An...
	6	AGENT TRUMAN	A Intrepid Panorama of a Robot And a Boy who...
	7	AIRPLANE SIERRA	A Touching Saga of a Hunter And a Butler who ...
	8	AIRPORT POLLOCK	A Epic Tale of a Moose And a Girl who must Con...
	9	ALABAMA DEVIL	A Thoughtful Panorama of a Database Administ...
	10	ALADDIN CALENDAR	A Action-Packed Tale of a Man And a Lumberjac...
	11	ALAMO VIDEOTAPE	A Boring Epistle of a Butler And a Cat who must ...
	12	ALASKA PHANTOM	A Fanciful Saga of a Hunter And a Pastry Chef ...
	13	ALI FOREVER	A Action-Packed Drama of a Dentist And a Croc...

film 15 ×

Output

Action Output

	#	Time	Action	Message
✓	138	14:43:36	SET SQL_SAFE_UPDATES = 1	0 row(s) affected
✓	139	14:44:18	SELECT film_id, title, replacement_cost FROM film WHERE replacement_...	0 row(s) returned
✓	140	14:49:25	SELECT c.customer_id, c.first_name, c.last_name, c.email, ...	7 row(s) returned
✓	141	14:53:17	SELECT customer_id, first_name, last_name, email FROM c...	599 row(s) returned
✓	142	14:56:42	SELECT film_id, title, description FROM film WHERE description ...	952 row(s) returned

- c) **Write a query to list all rentals from the rental table where return_date IS NULL**
checks for rentals where the return_date has no value, meaning the rental is still active or the movie has not been recorded as returned.

Query : **SELECT rental_id, rental_date, inventory_id, customer_id, staff_id, return_date**

FROM rental

WHERE return_date IS NULL;

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

Result Grid						
Filter Rows:						
Edit:						
Export/Import:						
Wrap Cell C						
	rental_id	rental_date	inventory_id	customer_id	staff_id	return_date
▶	11496	2006-02-14 15:16:03	2047	155	1	NULL
	11541	2006-02-14 15:16:03	2026	335	1	NULL
	11563	2006-02-14 15:16:03	1545	83	1	NULL
	11577	2006-02-14 15:16:03	4106	219	2	NULL
	11593	2006-02-14 15:16:03	817	99	1	NULL
	11611	2006-02-14 15:16:03	1857	192	2	NULL
	11646	2006-02-14 15:16:03	478	11	2	NULL
	11652	2006-02-14 15:16:03	1622	597	2	NULL
	11657	2006-02-14 15:16:03	3043	53	2	NULL
	11672	2006-02-14 15:16:03	3947	521	2	NULL
	11676	2006-02-14 15:16:03	4496	216	2	NULL
	11709	2006-02-14 15:16:03	1720	330	1	NULL
	11739	2006-02-14 15:16:03	4568	373	2	NULL
	11754	2006-02-14 15:16:03	3747	163	2	NULL

rental 17 ×

Output

Action Output

#	Time	Action	Message
✓ 140	14:49:25	SELECT c.customer_id, c.first_name, c.last_name, c.email, ...	7 row(s) returned
✓ 141	14:53:17	SELECT customer_id, first_name, last_name, email FROM c...	599 row(s) returned
✓ 142	14:56:42	SELECT film_id, title, description FROM film WHERE description ...	952 row(s) returned
✓ 143	14:59:58	SELECT rental_id, rental_date, inventory_id, customer_id, ...	183 row(s) returned

Q10. Operator Combination Challenge :

Category is NOT Children, Music, or Travel

rental_rate > 2.99

length is NOT BETWEEN 60 and 90

Query : SELECT f.title,

c.name AS category,

f.rental_rate,

f.length

FROM film f

JOIN film_category fc

ON f.film_id = fc.film_id

JOIN category c

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

```
ON fc.category_id = c.category_id  
WHERE c.name NOT IN ('Children', 'Music', 'Travel')  
AND f.rental_rate > 2.99  
AND f.length NOT BETWEEN 60 AND 90;
```



	title	category	rental_rate	length
▶	ACE GOLDFINGER	Horror	4.99	48
	AGENT TRUMAN	Foreign	3.44	169
	AIRPORT POLLOCK	Horror	4.99	54
	ALI FOREVER	Horror	5.74	150
	AMERICAN CIRCUS	Action	4.99	129
	ANTHEM LUKE	Comedy	4.99	91
	APACHE DIVINE	Family	4.99	92
	APOCALYPSE FLAMINGOS	New	4.99	119
	ARIZONA BANG	Classics	3.44	121
	ATTACKS HATE	Sci-Fi	4.99	113
	AUTUMN CROW	Games	4.99	108
	BABY HALL	Foreign	4.99	153
	BEAUTY GREASE	Drama	4.99	175
	BEHAVIOR RUNAWAY	Horror	5.74	100
	BIKINI BORROWERS	Animation	4.99	142
	BILKO ANONYMOUS	Family	4.99	100
	BILL OTHERS	Docume...	3.44	93

Result 18 x

Section D — Relationship Analysis: All Joins

Q11. INNER JOIN — Core Report

List each film along with its category and language. INNER JOIN connects the related tables using their matching IDs. ORDER BY f.title sorts the report alphabetically by film title

```
Query : SELECT f.title AS film_title,  
          c.name AS category_name,  
          l.name AS language_name  
FROM film AS f  
INNER JOIN film_category fc  
ON f.film_id = fc.film_id
```

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

INNER JOIN category c

ON fc.category_id = c.category_id

INNER JOIN language l

ON f.language_id = l.language_id ORDER BY f.title;

The screenshot shows a MySQL database interface. At the top, there's a toolbar with options like 'Result Grid', 'Filter Rows', 'Export', 'Wrap Cell Content', and 'Fetch rows'. Below this is a table with columns 'film_title', 'category_name', and 'language_name'. The table contains 12 rows of data. Below the table, there's a section labeled 'Result 20' with a close button. Underneath that is an 'Output' section with a dropdown menu set to 'Action Output'. This section displays a log of database actions, including the execution of the query shown in the previous blocks, with timestamps and row counts.

	film_title	category_name	language_name
▶	ACADEMY DINOSAUR	Documentary	English
	ACE GOLDFINGER	Horror	English
	ADAPTATION HOLES	Documentary	English
	AFFAIR PREJUDICE	Horror	English
	AFRICAN EGG	Family	English
	AGENT TRUMAN	Foreign	English
	AIRPLANE SIERRA	Comedy	English
	AIRPORT POLLOCK	Horror	English
	ALABAMA DEVIL	Horror	English
	ALADDIN CALENDAR	Sports	English
	ALAMO VIDEOTAPE	Foreign	English

#	Time	Action	Message
✓ 144	15:00:36	SELECT rental_id, rental_date, inventory_id, ...	183 row(s) returned
✓ 145	15:06:06	SELECT f.title, c.name AS category, f.rental_r...	263 row(s) returned
✓ 146	15:08:26	SELECT f.title, c.name AS category_name, l....	1000 row(s) returned
✓ 147	15:09:59	SELECT f.title AS film_title, c.name AS category_n...	1000 row(s) returned

Q12. Left & Right Join — Gap Analysis

- a) Using LEFT JOIN, list every customer along with their rental count — customers with zero rentals must still appear with a count of 0.

LEFT JOIN keeps all customers, even if they have no matching rental records. --
COUNT(r.rental_id) counts only the matching rental records. -- If a customer has no rentals, the count becomes 0. -- GROUP BY creates one result row for each customer.

Query : SELECT c.customer_id,

c.first_name,

c.last_name,

COUNT(r.rental_id) AS rental_count

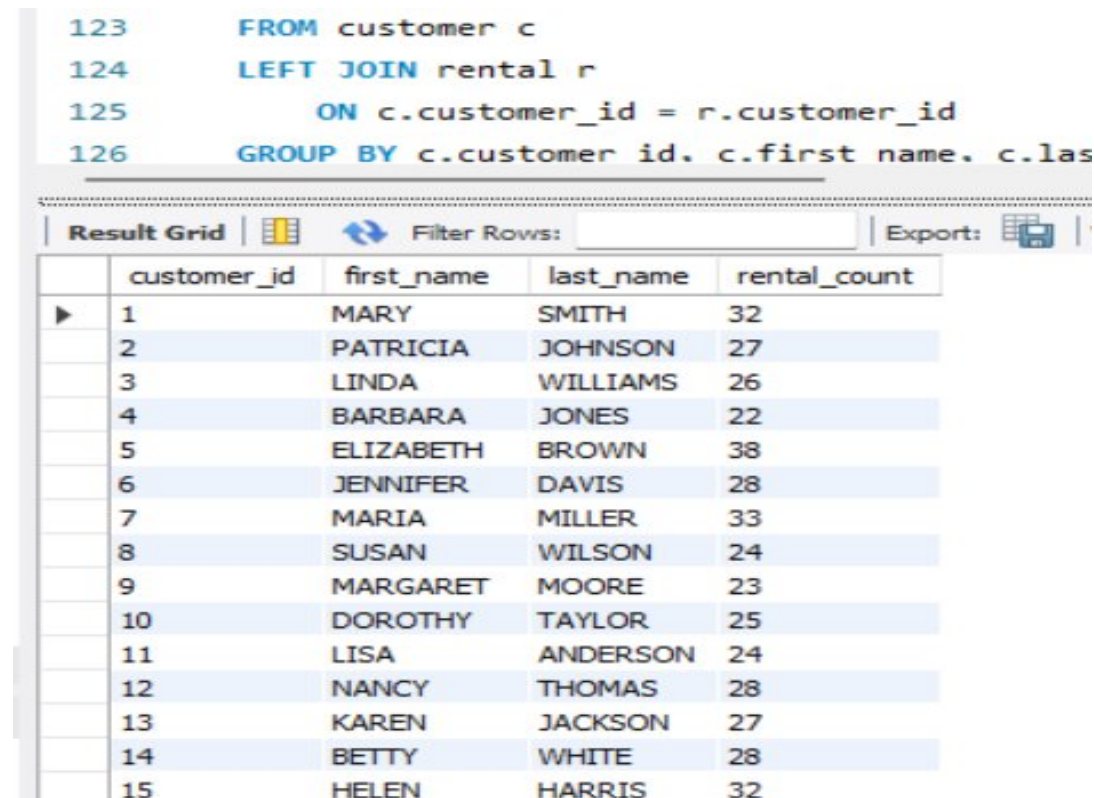
FROM customer c

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

LEFT JOIN rental r

ON c.customer_id = r.customer_id

GROUP BY c.customer_id, c.first_name, c.last_name;



The screenshot shows a SQL query editor with the following code:

```
123 FROM customer c
124 LEFT JOIN rental r
125     ON c.customer_id = r.customer_id
126 GROUP BY c.customer_id, c.first_name, c.last_name
```

Below the code is a 'Result Grid' showing the output of the query. The grid has columns for customer_id, first_name, last_name, and rental_count. The results are as follows:

	customer_id	first_name	last_name	rental_count
▶	1	MARY	SMITH	32
	2	PATRICIA	JOHNSON	27
	3	LINDA	WILLIAMS	26
	4	BARBARA	JONES	22
	5	ELIZABETH	BROWN	38
	6	JENNIFER	DAVIS	28
	7	MARIA	MILLER	33
	8	SUSAN	WILSON	24
	9	MARGARET	MOORE	23
	10	DOROTHY	TAYLOR	25
	11	LISA	ANDERSON	24
	12	NANCY	THOMAS	28
	13	KAREN	JACKSON	27
	14	BETTY	WHITE	28
	15	HELEN	HARRIS	32

b) Using RIGHT JOIN (rewrite the same logic from the opposite table order), produce the same result — write it starting FROM rental ... RIGHT JOIN customer.

RIGHT JOIN keeps all customers because customer is on the RIGHT side. --

COUNT(r.rental_id) counts the matching rental records. -- Customers with no rentals will have a count of 0. -- GROUP BY creates one result row for each customer.

Query : SELECT c.customer_id,

c.first_name,

c.last_name,

COUNT(r.rental_id) AS rental_count

FROM rental r

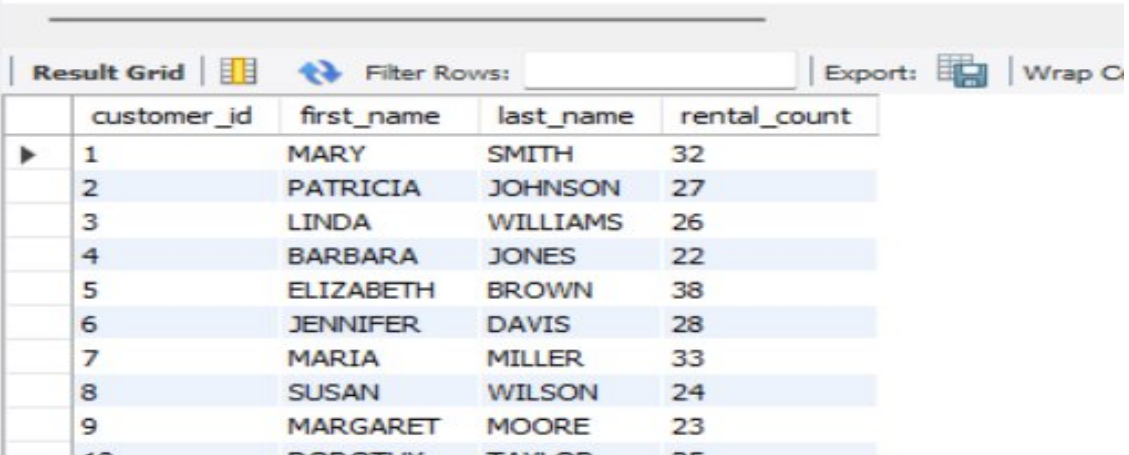
RIGHT JOIN customer c

ON r.customer_id = c.customer_id

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

GROUP BY c.customer_id, c.first_name, c.last_name;

```
133     RIGHT JOIN customer c
134         ON r.customer_id = c.customer_id
135     GROUP BY c.customer_id, c.first_name, c.last_name
136
```



	customer_id	first_name	last_name	rental_count
▶	1	MARY	SMITH	32
	2	PATRICIA	JOHNSON	27
	3	LINDA	WILLIAMS	26
	4	BARBARA	JONES	22
	5	ELIZABETH	BROWN	38
	6	JENNIFER	DAVIS	28
	7	MARIA	MILLER	33
	8	SUSAN	WILSON	24
	9	MARGARET	MOORE	23

Q13. FULL OUTER JOIN Simulation

Explanation : MySQL does not directly support FULL OUTER JOIN, so we combine LEFT JOIN + RIGHT JOIN using UNION.

- The first SELECT with LEFT JOIN gets all actors, including actors who are not connected to any film.
- The second SELECT with RIGHT JOIN gets all films, including films that are not connected to any actor.
- UNION combines both results and removes duplicate rows.
- CONCAT() combines the actor's first name and last name into one actor_name

Query : SELECT

a.actor_id,

CONCAT(a.first_name, ' ', a.last_name) AS actor_name,

f.film_id,

f.title

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

FROM actor a

LEFT JOIN film_actor fa

ON a.actor_id = fa.actor_id

LEFT JOIN film f

ON fa.film_id = f.film_id

UNION

SELECT

a.actor_id,

CONCAT(a.first_name, ' ', a.last_name) AS actor_name,

f.film_id,

f.title

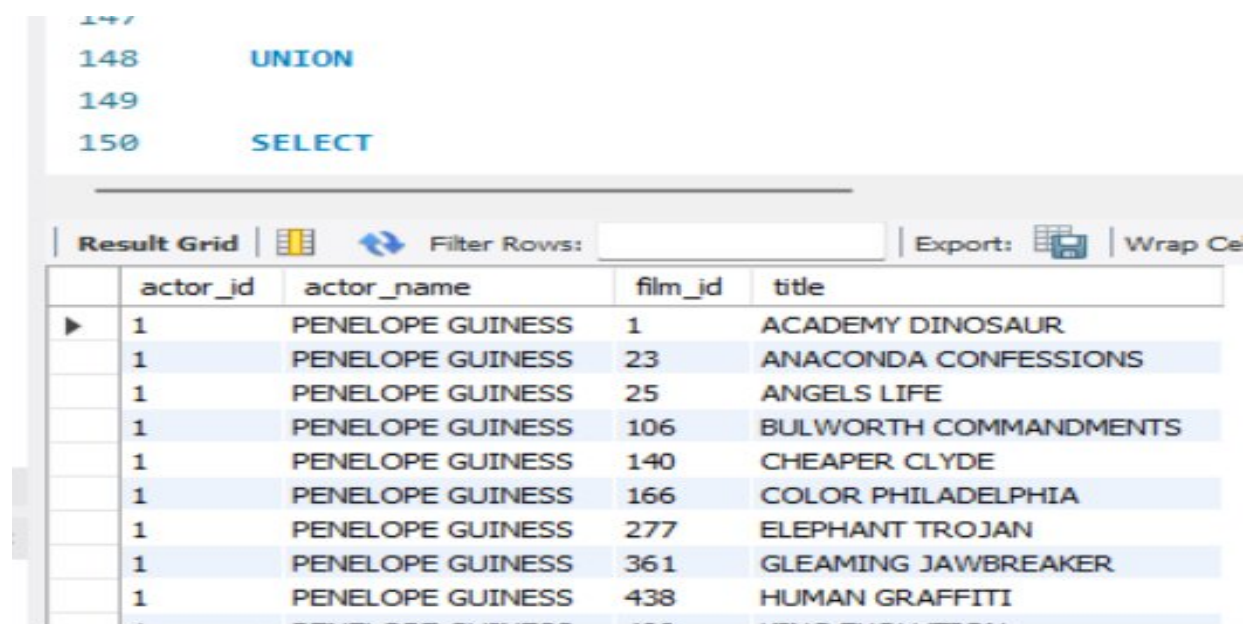
FROM actor a

RIGHT JOIN film_actor fa

ON a.actor_id = fa.actor_id

RIGHT JOIN film f

ON fa.film_id = f.film_id;



The screenshot shows a MySQL query editor with a SQL query and its results. The query is a UNION of two SELECT statements. The first SELECT statement selects actor_id, actor_name (concatenated first and last names), film_id, and title from actor, film_actor, and film tables. The second SELECT statement is identical but uses RIGHT JOIN. The results grid shows 10 rows of data, all with actor_id 1 and actor_name 'PENELOPE GUINESS'.

	actor_id	actor_name	film_id	title
▶	1	PENELOPE GUINESS	1	ACADEMY DINOSAUR
	1	PENELOPE GUINESS	23	ANACONDA CONFESSIONS
	1	PENELOPE GUINESS	25	ANGELS LIFE
	1	PENELOPE GUINESS	106	BULWORTH COMMANDMENTS
	1	PENELOPE GUINESS	140	CHEAPER CLYDE
	1	PENELOPE GUINESS	166	COLOR PHILADELPHIA
	1	PENELOPE GUINESS	277	ELEPHANT TROJAN
	1	PENELOPE GUINESS	361	GLEAMING JAWBREAKER
	1	PENELOPE GUINESS	438	HUMAN GRAFFITI
	1	PENELOPE GUINESS	400	THE FUGITIVE

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

Q14. SELF JOIN — Peer Comparison

The staff table is used twice with different aliases: s1 and s2, s1 represents the first staff member and s2 represents the second. s1.store_id = s2.store_id finds staff working in the same store. s1.staff_id < s2.staff_id prevents comparing a staff member with themselves -- and avoids getting the same pair twice. ORDER BY sorts the pairs by store and staff IDs.

Query : SELECT

```
s1.staff_id AS staff_1_id,  
s1.first_name AS staff_1_first_name,  
s1.last_name AS staff_1_last_name,  
s2.staff_id AS staff_2_id,  
s2.first_name AS staff_2_first_name,  
s2.last_name AS staff_2_last_name,  
s1.store_id
```

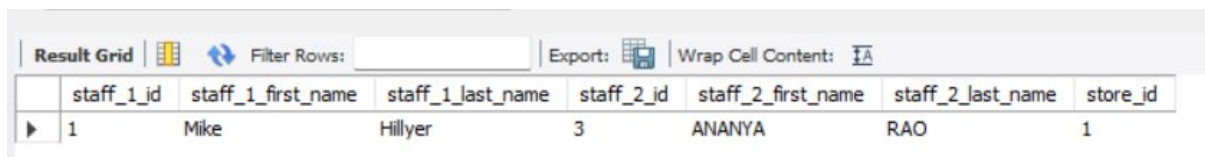
FROM staff AS s1

INNER JOIN staff AS s2

ON s1.store_id = s2.store_id

AND s1.staff_id < s2.staff_id

ORDER BY s1.store_id, s1.staff_id, s2.staff_id;



	staff_1_id	staff_1_first_name	staff_1_last_name	staff_2_id	staff_2_first_name	staff_2_last_name	store_id
▶	1	Mike	Hillyer	3	ANANYA	RAO	1

Q15. CROSS JOIN — Scenario Planning

Create every possible store-category combination. CROSS JOIN combines every store with every category. No matching condition is needed because all combinations are required. ORDER BY sorts the result first by store and then by category.

Query : SELECT

```
s.store_id,
```

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

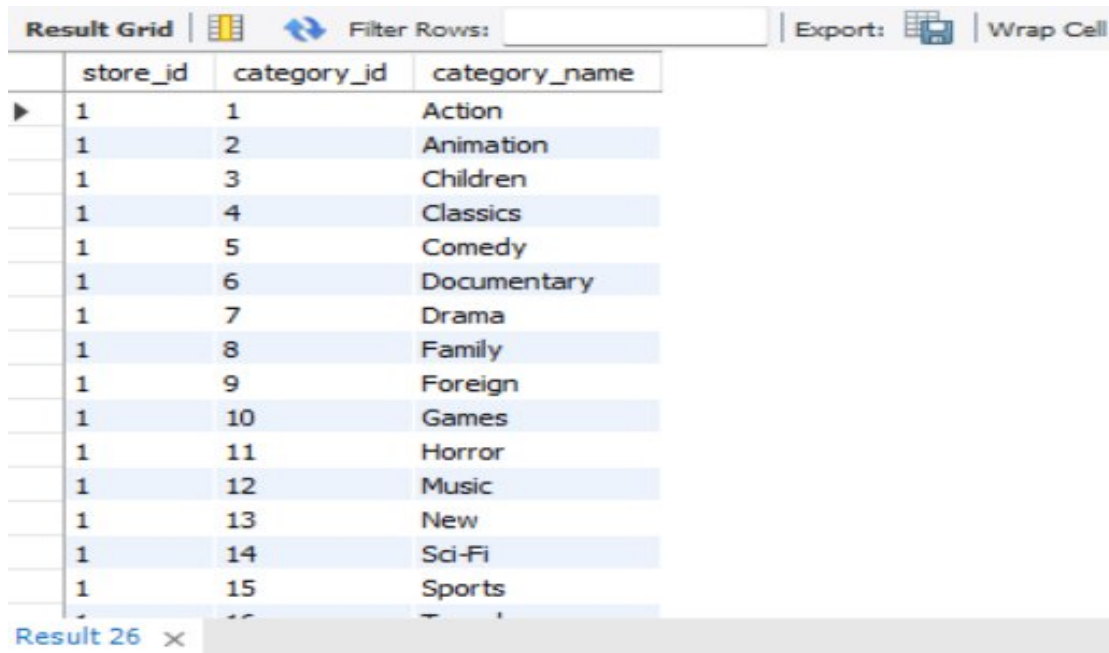
c.category_id,

c.name AS category_name

FROM store AS s

CROSS JOIN category AS c

ORDER BY s.store_id, c.category_id;



The screenshot shows a MySQL Result Grid window. The title bar includes 'Result Grid', a grid icon, a 'Filter Rows:' dropdown, an 'Export:' button with a document icon, and a 'Wrap Cell' button. The table has three columns: 'store_id', 'category_id', and 'category_name'. The data is as follows:

store_id	category_id	category_name
1	1	Action
1	2	Animation
1	3	Children
1	4	Classics
1	5	Comedy
1	6	Documentary
1	7	Drama
1	8	Family
1	9	Foreign
1	10	Games
1	11	Horror
1	12	Music
1	13	New
1	14	Sci-Fi
1	15	Sports

At the bottom left of the window, it says 'Result 26' with a close button (X).

Section E — Aggregation, Grouping & Output-Based Questions

Q16. Revenue Report — Grouped

- a) Write a query using GROUP BY to show total revenue (SUM(amount)) per staff_id from payment.

Calculate the total revenue collected by each staff member. SUM(amount) adds all payment amounts handled by that staff member. GROUP BY staff_id creates a separate total for each staff member.

Query : SELECT

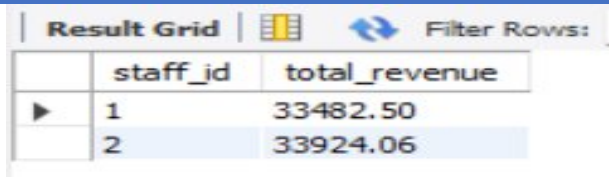
staff_id,

SUM(amount) AS total_revenue

FROM payment

GROUP BY staff_id;

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

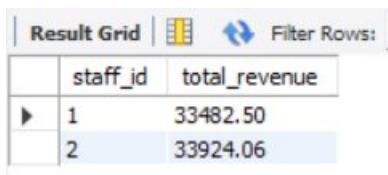


	staff_id	total_revenue
▶	1	33482.50
	2	33924.06

b) Extend it with HAVING to show only staff whose total revenue exceeds 15000.

Calculate the total revenue for each staff member. -- GROUP BY creates one group for each staff member. -- SUM(amount) adds all payments handled by each staff member. -- HAVING keeps only staff whose total revenue is greater than 15000.

Query : SELECT
 staff_id,
 SUM(amount) AS total_revenue
FROM payment
GROUP BY staff_id
HAVING SUM(amount) > 15000;



	staff_id	total_revenue
▶	1	33482.50
	2	33924.06

c) Now write a version without grouping — a single query showing the overall company-wide total revenue and the single highest individual payment amount, side by side .

Calculate the overall revenue generated by the company. SUM(amount) adds all payment amounts from the payment table. MAX(amount) finds the highest single payment made. No GROUP BY is used because we need one overall result for the entire company.

Query : SELECT
 SUM(amount) AS total_company_revenue,
 MAX(amount) AS highest_individual_payment
FROM payment;



	total_company_revenue	highest_individual_payment
▶	67406.56	11.99

Q17. Output-Based Question — Reverse Engineer the Query

Show the number of films and average rental rate for each category. INNER JOIN connects category, film_category, and film tables. COUNT(film_id) counts how many films belong to each category. AVG(rental_rate) calculates the average rental rate for each category.

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

ROUND(..., 2) keeps the average rate to 2 decimal places. HAVING keeps only categories having more than 60 films. ORDER BY sorts the categories from highest film count to lowest.

Query : SELECT

c.name AS category_name,

COUNT(f.film_id) AS total_films,

ROUND(AVG(f.rental_rate), 2) AS avg_rental_rate

FROM category AS c

INNER JOIN film_category AS fc

ON c.category_id = fc.category_id

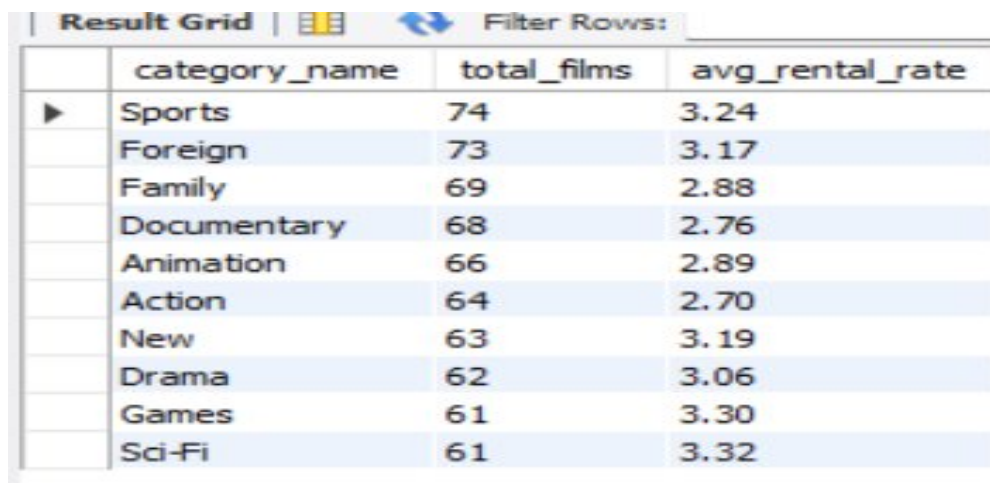
INNER JOIN film AS f

ON fc.film_id = f.film_id

GROUP BY c.category_id, c.name

HAVING COUNT(f.film_id) > 60

ORDER BY total_films DESC;



	category_name	total_films	avg_rental_rate
▶	Sports	74	3.24
	Foreign	73	3.17
	Family	69	2.88
	Documentary	68	2.76
	Animation	66	2.89
	Action	64	2.70
	New	63	3.19
	Drama	62	3.06
	Games	61	3.30
	Sci-Fi	61	3.32

Q18. Output-Based Question — Rank the Customers

Calculate how much each customer has spent. SUM(p.amount) adds all payments made by each customer. GROUP BY creates one result for each customer. DENSE_RANK() ranks customers from highest total spending to lowest. Customers with the same spending receive the same rank. ORDER BY spend_rank displays the highest spender first.

Query : SELECT

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

```
c.customer_id,  
c.first_name,  
SUM(p.amount) AS total_spent,  
DENSE_RANK() OVER (  
    ORDER BY SUM(p.amount) DESC  
) AS spend_rank  
FROM customer AS c  
INNER JOIN payment AS p  
    ON c.customer_id = p.customer_id  
GROUP BY c.customer_id, c.first_name  
ORDER BY spend_rank;
```



The screenshot shows a MySQL query result grid with the following columns: customer_id, first_name, total_spent, and spend_rank. The results are ordered by total_spent in descending order, with Karl (customer_id 526) having the highest total spent at 221.55, ranked 1st. The grid includes a toolbar at the top with options for 'Result Grid', 'Filter Rows', and 'Export'. The bottom of the grid shows 'Result 31'.

	customer_id	first_name	total_spent	spend_rank
▶	526	KARL	221.55	1
	148	ELEANOR	216.54	2
	144	CLARA	195.58	3
	137	RHONDA	194.61	4
	178	MARION	194.61	4
	459	TOMMY	186.62	5
	469	WESLEY	177.60	6
	468	TIM	175.61	7
	236	MARCIA	175.58	8
	181	ANA	174.66	9
	176	JUNE	173.63	10
	50	DIANE	169.65	11
	259	LENA	168.68	12
	522	ARNOLD	167.67	13
	410	CURTIS	167.62	14

Section F — Advanced Analytics: Subqueries & Window Functions

Q19. Subqueries — Nested & Correlated

- a) Nested subquery: Write a query to find all films whose rental_rate is higher than the average rental rate of their own category

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

Find films whose rental rate is higher than the average rental rate of their own category. The outer query gets each film and its category. The subquery calculates the average rental rate for that category. -- The category_id from the outer query is used inside the subquery -- to calculate the correct category-wise average.

Query : SELECT

```
f.film_id,  
f.title,  
f.rental_rate,  
fc.category_id
```

FROM film AS f

INNER JOIN film_category AS fc

```
ON f.film_id = fc.film_id
```

WHERE f.rental_rate > (

```
SELECT AVG(f2.rental_rate)
```

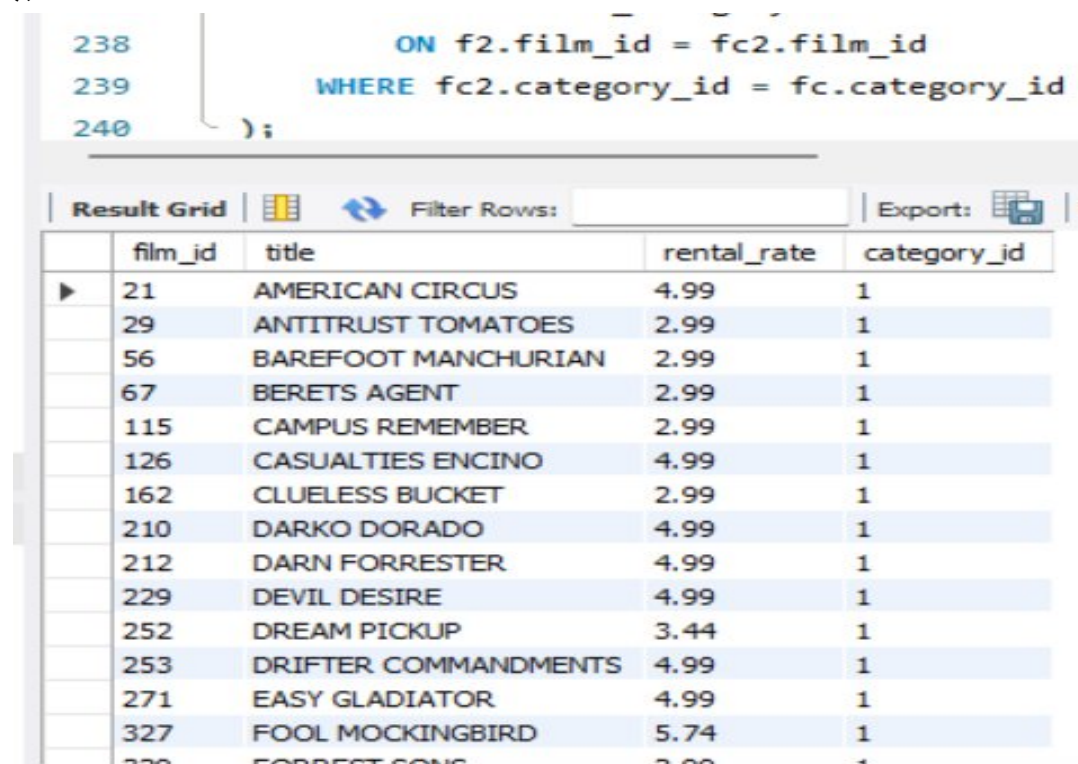
```
FROM film AS f2
```

```
INNER JOIN film_category AS fc2
```

```
ON f2.film_id = fc2.film_id
```

```
WHERE fc2.category_id = fc.category_id
```

);



The screenshot shows a MySQL query editor with the following SQL code:

```
238         ON f2.film_id = fc2.film_id  
239     WHERE fc2.category_id = fc.category_id  
240 );
```

Below the query editor is a 'Result Grid' showing the results of the query. The grid has columns for film_id, title, rental_rate, and category_id. The results are as follows:

	film_id	title	rental_rate	category_id
▶	21	AMERICAN CIRCUS	4.99	1
	29	ANTITRUST TOMATOES	2.99	1
	56	BAREFOOT MANCHURIAN	2.99	1
	67	BERETS AGENT	2.99	1
	115	CAMPUS REMEMBER	2.99	1
	126	CASUALTIES ENCINO	4.99	1
	162	CLUELESS BUCKET	2.99	1
	210	DARKO DORADO	4.99	1
	212	DARN FORRESTER	4.99	1
	229	DEVIL DESIRE	4.99	1
	252	DREAM PICKUP	3.44	1
	253	DRIFTER COMMANDMENTS	4.99	1
	271	EASY GLADIATOR	4.99	1
	327	FOOL MOCKINGBIRD	5.74	1
	328	FOURTH CORNER	2.99	1

b) EXISTS subquery: Write a query to list customers who have never made a payment

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

Find customers who have never made a payment. -- NOT EXISTS checks that no matching payment record exists -- for the current customer. -- The subquery matches payment.customer_id with customer.customer_id.

Query : SELECT

c.customer_id,
c.first_name,
c.last_name

FROM customer AS c

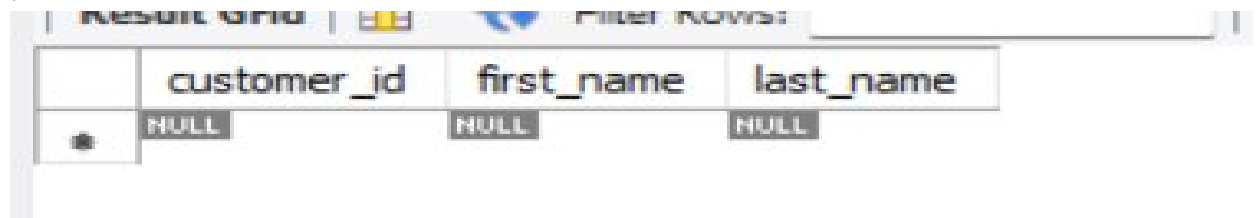
WHERE NOT EXISTS (

SELECT 1

FROM payment AS p

WHERE p.customer_id = c.customer_id

);



	customer_id	first_name	last_name
1	NULL	NULL	NULL

- c) explain the performance difference between using IN with a subquery versus EXISTS when the subquery returns a large number of rows.

Explanation : When the subquery returns a large number of rows:

IN → may need to generate and compare against a large list of values, which can use more memory/work.

EXISTS → checks whether a matching row exists and can often stop as soon as it finds the first match, making it more efficient for large datasets.

Q20. Window Functions — Store-Level Analysis

- a) Write a query using ROW_NUMBER() (or RANK()) partitioned by store_id and ordered by rental count descending, then filter to only the top 3 per store

Query : SELECT

c.customer_id,
c.first_name,
c.last_name

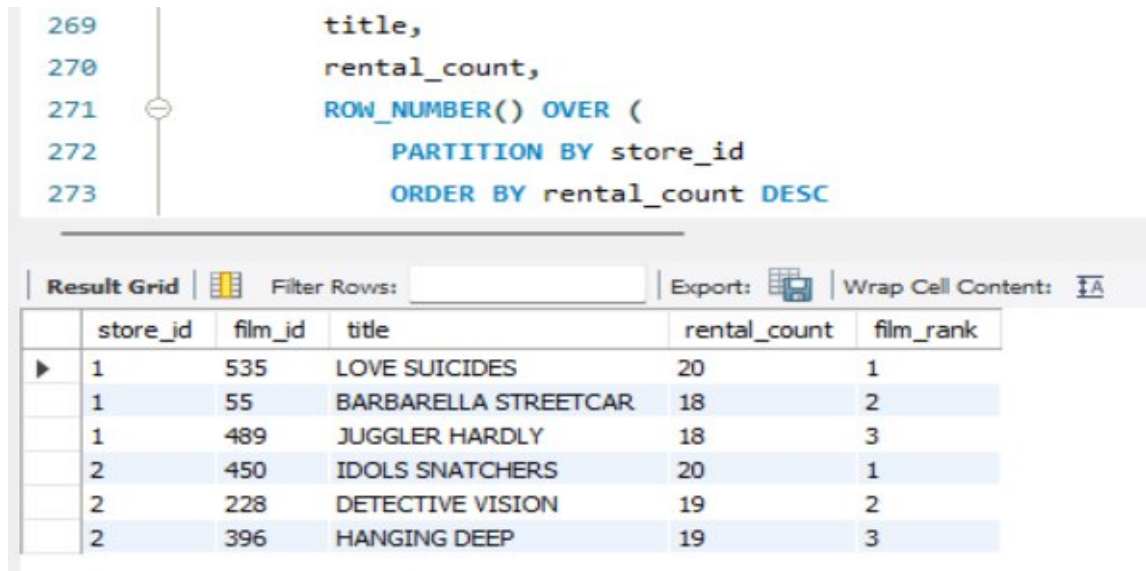
FROM customer AS c

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

```
WHERE NOT EXISTS (  
    SELECT 1  
    FROM payment AS p  
    WHERE p.customer_id = c.customer_id  
);  
WITH film_rental_counts AS (  
    SELECT  
        i.store_id,  
        f.film_id,  
        f.title,  
        COUNT(r.rental_id) AS rental_count  
    FROM inventory AS i  
    INNER JOIN film AS f  
        ON i.film_id = f.film_id  
    LEFT JOIN rental AS r  
        ON i.inventory_id = r.inventory_id  
    GROUP BY i.store_id, f.film_id, f.title  
),  
ranked_films AS (  
    SELECT  
        store_id,  
        film_id,  
        title,  
        rental_count,  
        ROW_NUMBER() OVER (  
            PARTITION BY store_id  
            ORDER BY rental_count DESC  
        ) AS film_rank  
    FROM film_rental_counts  
)  
SELECT  
    store_id,  
    film_id,  
    title,
```

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

```
rental_count,  
film_rank  
FROM ranked_films  
WHERE film_rank <= 3  
ORDER BY store_id, film_rank;
```



The screenshot shows a SQL query editor with a query window on the left and a result grid on the right. The query is as follows:

```
269 title,  
270 rental_count,  
271 ROW_NUMBER() OVER (  
272     PARTITION BY store_id  
273     ORDER BY rental_count DESC
```

The result grid displays the following data:

	store_id	film_id	title	rental_count	film_rank
▶	1	535	LOVE SUICIDES	20	1
	1	55	BARBARELLA STREETCAR	18	2
	1	489	JUGGLER HARDLY	18	3
	2	450	IDOLS SNATCHERS	20	1
	2	228	DETECTIVE VISION	19	2
	2	396	HANGING DEEP	19	3

Explanation

This query is used to **find the top 3 most-rented films in each store.**

- First, film_rental_counts calculates the **number of rentals for each film in each store.**
- ROW_NUMBER() then gives a **rank to each film separately within each store.**
- PARTITION BY store_id means the ranking **starts again for every store.**
- ORDER BY rental_count DESC puts the film with the highest rental count at **Rank 1.**
- Finally, WHERE film_rank <= 3 selects only the **top 3 films from each store.**
- ORDER BY store_id, film_rank displays the results store-wise and rank-wise.

b) explain the difference between ROW_NUMBER(), RANK(), and DENSE_RANK() if two films in the same store had the exact same rental count.

Explanation : Difference Between ROW_NUMBER(), RANK() and DENSE_RANK()

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

- ROW_NUMBER() gives each film a different rank, even when two films have the same rental count.
- RANK() gives the same rank to films with the same rental count, but skips the next rank.
- DENSE_RANK() gives the same rank to films with the same rental count, but does not skip the next rank.

Example: If two films have the same rental count:

ROW_NUMBER() → 1, 2, 3

RANK() → 1, 2, 2, 4

DENSE_RANK() → 1, 2, 2, 3

Optional :

Write a single query combining a CTE, a JOIN, and a window function to show each customer's rental history along with a running total (SUM() OVER) of their spending, ordered by payment date.

Query : WITH rental_history AS (

SELECT

c.customer_id,
c.first_name,
c.last_name,
r.rental_id,
r.rental_date,
p.payment_date,
p.amount

FROM customer AS c

INNER JOIN rental AS r

ON c.customer_id = r.customer_id

INNER JOIN payment AS p

ON r.rental_id = p.rental_id

)

SELECT

customer_id,
first_name,
last_name,

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

```
rental_id,  
rental_date,  
payment_date,  
amount,  
SUM(amount) OVER (  
    PARTITION BY customer_id  
    ORDER BY payment_date  
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
) AS running_total  
FROM rental_history  
ORDER BY customer_id, payment_date;
```

```
301 ~ )  
302 SELECT  
303     customer_id,  
304     first_name,  
305     last_name,
```

	customer_id	first_name	last_name	rental_id	rental_date	payment_date	amount	running_total
▶	1	MARY	SMITH	76	2005-05-25 11:30:37	2005-05-25 11:30:37	2.99	2.99
	1	MARY	SMITH	573	2005-05-28 10:35:23	2005-05-28 10:35:23	0.99	3.98
	1	MARY	SMITH	1185	2005-06-15 00:54:12	2005-06-15 00:54:12	5.99	9.97
	1	MARY	SMITH	1422	2005-06-15 18:02:53	2005-06-15 18:02:53	0.99	10.96
	1	MARY	SMITH	1476	2005-06-15 21:08:46	2005-06-15 21:08:46	9.99	20.95
	1	MARY	SMITH	1725	2005-06-16 15:18:57	2005-06-16 15:18:57	4.99	25.94
	1	MARY	SMITH	2308	2005-06-18 08:41:48	2005-06-18 08:41:48	4.99	30.93
	1	MARY	SMITH	2363	2005-06-18 13:33:59	2005-06-18 13:33:59	0.99	31.92
	1	MARY	SMITH	3284	2005-06-21 06:24:45	2005-06-21 06:24:45	3.99	35.91
	1	MARY	SMITH	4526	2005-07-08 03:17:05	2005-07-08 03:17:05	5.99	41.90
	1	MARY	SMITH	4611	2005-07-08 07:33:56	2005-07-08 07:33:56	5.99	47.89
	1	MARY	SMITH	5244	2005-07-09 13:24:07	2005-07-09 13:24:07	4.99	52.88

Explanation

- The **CTE (rental_history)** first combines the customer, rental, and payment tables to get each customer's rental and payment details.
- The **JOINS** connect the customer with their rentals and the corresponding payments.
- SUM(amount) OVER calculates the **running total of spending** for each customer.
- PARTITION BY customer_id makes the running total **start separately for each customer**.
- ORDER BY payment_date adds each payment to the running total in **payment-date order**.

MYSQL ASSISSNMENT – SAKILA DATA ANALYSIS

- ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW means the total starts from the customer's **first payment and continues up to the current payment**.
- The final ORDER BY customer_id, payment_date displays each customer's history in chronological order.